Deployment Strategy for Feynman-Pure Architecture and Mathematical Text Compression within Stela Governance

1. Analysis and Discussion

The proposition to integrate a "Feynman-Pure Architecture" with "Mathematical Text Compression for AI" establishes a profound paradigm shift in how autonomous and semi-autonomous systems govern state, process directives, and execute repository maintenance. The overarching objective is to mathematically distill linguistic and operational inputs into the highest possible density of "signal," aggressively eliminating the ambient grammatical and stylistic "noise" inherent in human language. This compressed, high-entropy signal is then processed through rigid analogical reasoning frameworks to ensure flawless execution without contextual bloat. When applied to a strictly governed, pointer-first repository framework such as the Stela ecosystem, this theoretical synthesis presents a pathway to eliminate semantic drift, maximize context window efficiency, and enforce the absolute operational determinism mandated by the Policy of Truth (PoT.md).

The following analysis deconstructs both the mathematical foundations of text compression and the cognitive architecture of the Feynman technique, synthesizes them into a singular operational framework termed the "Feynman-Shannon Paradigm," and rigorously evaluates the precise intersection of this synthesis with Stela's immutable workflow, automated enforcement gates, and heuristic scripts.

1.1 Mathematical Foundations of Text Information Density

The foundational premise of textual compression for machine learning systems relies heavily on Claude Shannon's Information Theory, which fundamentally altered the understanding of communication by quantifying information as a mathematical resolution of uncertainty.[1] Language, by its evolutionary nature, contains significant redundancy. This redundancy, consisting of filler words, highly predictable syntactic structures, and stylistic padding, consumes finite computational resources (tokens, memory, and processing cycles) without providing new logical value.[1]

In information theory, the Shannon entropy of a discrete random variable quantifies the expected value of the information—or "surprise"—contained in a message.[3] The mathematical goal of text compression for artificial intelligence is to represent the maximum amount of true information using the absolute minimum number of discrete tokens.[1] Given a discrete random variable X with possible outcomes x situated within the state space $\mathcal{X}$, the entropy is

defined mathematically as:

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

This equation dictates that events or symbols with high probability convey exceptionally low information, whereas rare events convey high information.[3] An equivalent definition of entropy is the expected value of the self-information of a variable.[3] To translate this to natural language processing, a discrete information source operates similarly to a Markoff process; for each state i, there exists a set of probabilities $p_i(j)$ of producing subsequent symbols j.[6] If the Markoff process proceeds at a definite time rate, the entropy per symbol of text measures the constraint imposed on the language due to its statistical structure.[6]

A "Feynman-Pure" system posits that a model's intelligence is directly correlated to its ability to compress information without losing the underlying logical fidelity.[1] Perfect compression requires a model to deeply understand the relational patterns, underlying logic, and world knowledge embedded within a text so that it can discard the predictable elements (noise) and retain only the core axioms (signal).[1]

1.2 Algorithmic Mechanisms for Semantic Distillation

To practically deploy the principles of information theory into a computational pipeline, specific mathematical reduction techniques must be utilized to convert standard operational text into high-density instructional payloads. The optimization of semantic density involves mapping textual data into multidimensional spaces to isolate and extract core logic.

1.2.1 Geodesic Distances and Hypersphere Mapping

Advanced semantic density frameworks represent semantic information as points on a high-dimensional hypersphere $\mathbb{S}^n$ with a defined radius r.[8] This geometrical representation allows the system to capture the intricate relationships between different semantic states. Within this space, core clusters represent dense regions of semantic points characterized by a centroid and a density function.[8] The geodesic distance d_g between two points on the hypersphere measures the shortest path between them, facilitating the modeling of semantic transitions.[8] By utilizing Sobolev dot products, the system can measure the interaction between functions representing semantic states, capturing the integration of information before projecting it onto a Riemann manifold to analyze long-term dependencies.[8]

1.2.2 Primary Reduction Techniques

Operating upon these geometrical and probabilistic foundations, several distinct algorithms

execute the actual compression of the text [1]:

- **Byte Pair Encoding (BPE):** An iterative algorithmic replacement technique that identifies the most frequent character sequences and replaces them with single tokens. This operates at the sub-word level to physically shrink the data footprint.
- **Latent Semantic Analysis (LSA):** A method utilizing Singular Value Decomposition (SVD) to map text into a dense vector space. A term-document matrix M is decomposed into three matrices: $U\Sigma V^T$. By truncating the smallest singular values in the diagonal matrix Σ, the system discards the dimensions that mathematically represent statistical noise, retaining only the dominant, high-variance semantic structures.[1]
- **Semantic Distillation:** The programmatic removal of "low-entropy" vocabulary. By analyzing the contextual probability $p(x)$ of each word, the algorithm discards predictable linguistic filler (e.g., "the," "extremely," "in order to") to isolate the core operational verbs and relational nouns without altering the instructional imperative.[1]
- **Prompt Compression:** Utilizing specialized attention-aware algorithms such as LLMLingua or Selective Context, this technique actively identifies and strips "low-attention" tokens.[1] These are specific tokens that the large language model's (LLM) internal attention mechanism would mathematically ignore or assign near-zero weight to during the forward pass.[1]

The empirical application of these techniques results in a quantifiable increase in semantic density. High semantic density ensures that the AI model focuses exclusively on critical relationships, actively preventing "attention drift" over extensive context windows.[1] Studies indicate that these combinations of algorithmic reductions can reduce textual volume by 50% to 70%, profoundly increasing the context efficiency of the model, accelerating inference times, and reducing operational token expenditures.[1]

1.3 The Feynman Architecture and Analogical NLP

While Shannon entropy provides the mathematics for compression, the "Feynman Mode" provides the cognitive architecture for reconstruction and learning. The Feynman pedagogical technique relies on breaking down highly complex subjects into their most fundamental, axiomatic truths, aggressively stripping away obfuscating jargon, and utilizing precise analogical reasoning to verify true comprehension.[10]

In the realm of Natural Language Processing (NLP), analogy-making gives rise to reasoning, abstraction, and counter-factual inference—capabilities critical for engineering non-brittle AI systems that can robustly generalize across novel domains.[13] Cognitive theories of analogy differentiate between surface similarities, relational similarities, and deep system similarities.[14] Existing NLP research frequently struggles with complex system analogies, often relying on

basic word-pair relations rather than mapping the underlying architecture of procedural texts.[13]

A Feynman-Pure AI architecture must overcome this by extracting entities and relations from a text and finding mathematical mappings between different domains based entirely on relational similarity.[13] For instance, understanding a procedural analogy requires the system to infer information not explicitly mentioned in the target domain by transforming the semantic topology of the source domain.[15] Recent advancements in Large Language Models (LLMs) indicate that guiding models through solved examples and Chain-of-Thought reasoning can enhance this analogical detection [14], but true Feynman-level mastery requires the AI to independently generate heuristics that evaluate system-level mappings.[16]

The Evolution of Heuristics (EoH) paradigm demonstrates that LLMs can dynamically generate novel program code to evaluate complex combinatorial optimization problems.[16] By combining EoH with analogical reasoning, a Feynman-Pure system does not merely read instructions; it maps the instructions to known historical success states, stripping away the surface-level text to operate purely on the mathematical relations of the task.

1.4 Architectural Intersection with Stela Governance

The Stela ecosystem is a highly regulated, repo-resident AI framework governed strictly by a singular constitutional document, PoT.md.[1] The operational philosophy of Stela is defined by its Axioms, which state that "deception or incoherence are inherently destabilizing and must be resolved," and that "Truth" is a primitive predicate governed by specific rules rather than a mere definition.[1] Evaluating the Feynman-Shannon Paradigm against Stela's physical laws reveals a high degree of ideological alignment regarding precision and efficiency, but it also exposes significant operational friction points regarding human relatability and syntax validation.

1.4.1 Alignment with Linguistic Precision and Anti-Drift Mandates

Stela strictly enforces a "Linguistic Precision" mandate that prohibits contractions across any scope, requires quantitative reporting for deviations from protocol, and demands absolute literalism.[1] Furthermore, Stela operates an Integrator/Contractor staffing protocol where the Integrator AI maintains structural integrity while the Contractor AI executes logic tasks.[1]

The Feynman-Shannon Paradigm is the logical, mathematical evolution of Stela's Linguistic Precision mandate. By defining semantic noise as computational incoherence, mathematical text compression provides a quantitative method to enforce the total absence of ambiguity. In PoT.md, ambiguity is explicitly defined as an instruction capable of multiple interpretations, triggering a System Failure State and a "STOP WORK" mandate.[1] Semantic distillation actively neutralizes ambiguity by stripping the vocabulary that causes multiple interpretations, leaving only high-entropy logical operators.[1]

However, traditional readability metrics currently utilized in the broader industry often conflict

with AI-optimized text density.

Metric Paradigm	Calculation Mechanism	AI Optimization Value	Conflict with Feynman-Shannon
Flesch-Kincaid	Evaluates word length (syllables) and sentence length (words per sentence).[19]	Highly limited. Lower scores (higher reading grade) often correlate with complex technical jargon, but not necessarily high semantic density.[19]	Encourages simpler, longer sentences with low entropy. Actively penalizes dense, mathematically precise system analogies.[21]
Gunning Fog Index	Accounts for sentence length and the percentage of complex words (three or more syllables).[20]	Poor. Polysyllabic words do not inherently increase mathematical information density; they often introduce stylistic noise.	Punishes necessary technical axioms while rewarding monosyllabic filler.
Prompt-Based Metrics	LLM prompts exploit general language understanding to classify abstract text features.[22]	Moderate to High. Captures signals beyond static metrics.[22]	Computationally expensive to run concurrently with execution paths.
Semantic Density (SVD/Entropy)	Projects semantics onto a hypersphere; calculates geodesic distance and token predictability.[1]	Maximum. Directly reduces context window footprint and isolates relational logic.[1]	The resulting text may appear grammatically broken to a human reader, violating Stela's Relatability Mandate.[1]

Stela's current linter, tools/lint/style.sh, enforces anti-jargon policies via a static JARGON_BLACKLIST and uses regex to block contractions.[1] This is a primitive form of semantic

distillation. Transitioning from regex-based style enforcement to SVD-based entropy enforcement represents a massive leap in capability.

1.4.2 Impact on the Immutable DP Workflow

Stela operates on an Immutable Dispatch Packet (DP) workflow, centrally executed via the TASK.md contract.[1] The sequence involves invoking ops/bin/open to establish a verifiable freshness state, running ops/bin/draft to generate canonical DP structures from ops/src/surfaces/dp.md.tpl, executing the task, and finally running ops/bin/certify to generate RESULTS receipts.[1]

The DP structures rely on exact hashing. The tools/lint/dp.sh linter enforces a canonical template hash parity check; it normalizes both canonical and payload structures, hashes them, and fails on mismatch.[1] If a text compression algorithm processes the DP and strips structural boilerplate or heading ID formatting to save tokens, the normalized structure hash will instantly fail, causing tools/lint/dp.sh to trigger a hard stop.[1]

Therefore, applying the synthesis to this workflow implies that the payloads traversing from storage/dp/intake/ to the Contractor AI must undergo compression *without* mutating the underlying tracked files. A prompt drafted by an Operator might state: *"The Contractor must meticulously analyze the git status and ensure that no uncommitted changes exist in the main branch before proceeding."* A Feynman-Shannon distiller would reduce this to: *"Contractor evaluate git status: main branch uncommitted changes = STOP"*.[1] This distillation reduces the context payload significantly, fundamentally decreasing the latency of Contractor execution while preserving 100% of the logical intent required to pass Stela's CI/CD gates.[1]

1.4.3 Context Rehydration and Heuristic Expansion

Stela utilizes a robust, pointer-first context rehydration architecture. The manifest file ops/lib/manifests/CONTEXT.md defines required artifacts, and ops/bin/context combined with ops/lib/scripts/synthesize.sh builds the session stream by compiling llms-core.txt and llms-full.txt.[1]

The synthesis engine (synthesize.sh) is the mathematically optimal injection point for the Feynman-Shannon architecture. Currently, synthesize.sh parses manifests, enforces hazards, and emits formatted streams.[1] Concurrently, ops/bin/llms generates these discovery entry points to keep the system synchronized with manifest truth, preventing SSOT drift.[1] If this engine is upgraded to calculate the Shannon entropy of incoming text and filter tokens below a calculated semantic density threshold [8], the llms.txt payloads provided to the AI will transition from standard human-readable documentation into mathematically pure logic vectors.

Furthermore, Stela employs an advanced heuristic engine, ops/lib/scripts/heuristics.sh, which calculates Pattern Density and hot zone forensics using git diff and git log metrics, detecting semantic collisions via token overlap frequency.[1] To fully deploy a Feynman-Pure architecture,

this heuristic engine must be expanded to include analogical evaluation.[23] When the Integrator AI drafts a new DP, the analogical heuristic would calculate the geodesic distance [8] between the current task and historical data in archives/surfaces/PoW-*.md (Proof of Work) and SoP.md (State of Play). It would then synthesize historical success parameters into the current DP, achieving the PoT.md goal of "Reuse-first" [1] through high-order algorithmic reasoning rather than literal string matching.

1.5 Systemic Risks and Friction Contexts

Deploying mathematical text compression into a repository governed by severe file schema rules and automated bash scripts presents highly specific systemic risks. The integration must navigate the narrow corridor between token optimization and repository corruption.

Failure Mode	Mechanism of Failure	Architectural Consequence	System Response
Schema Hash Mismatch	tools/lint/dp.sh enforces canonical template hashing by comparing ops/src/surfaces/dp.md.tpl against TASK.md.[1]	If the compression algorithm strips structural boilerplate from TASK.md, the normalized structure hash will fail the CI gate.	**STOP WORK.** CI gates (.github/workflows/gates.yml) reject the PR.[1]
Loss of Operator Explainability	Over-compression (e.g., severe BPE or extreme LSA truncation) removes human-readable syntax.[1]	The RESULTS artifact becomes an unreadable matrix of high-entropy terms, violating the "Relatability Mandate" in PoT.md.[1]	Equilibrium failure; Human Operator cannot verify state impact and refuses to merge.[1]
Pointer Truncation	Distillation algorithms frequently strip backticks, hyphens, or non-standard syntax.[1]	If file paths (e.g., `ops/bin/certify`) are corrupted in the text, tools/lint/context.sh will fail artifact reachability	**STOP WORK.** Missing required artifacts error.[1]

		checks.[1]	
Missing Leaf Telemetry	tools/lint/leaf.sh ensures all execution scripts emit emit_binary_leaf tokens.[1]	If the Feynman distiller operates externally and bypasses telemetry, the proof ledger cannot trace the modification.	Unlogged execution paths violate PoW.md continuity.[1]

Consequently, the strategic deployment must rigidly separate the **Execution Stream** (the compressed, high-density data fed ephemerally to the AI Contractor) from the **Canon Surfaces** (the human-readable, schema-compliant markdown files committed to the repository's main branch).

---

2. Strategic Options

To successfully deploy the Feynman-Shannon Synthesis into Stela while adhering to the absolute, non-negotiable constraints of PoT.md, the integration must be highly deterministic, entirely auditable through RESULTS artifacts, and computationally incapable of triggering unresolvable state drift.

The following strategic options present three distinct architectural paths for deploying this paradigm. They range from runtime pipeline modifications to hard repository gates, concluding with an autonomous agent-driven approach.

2.1 Option A: Ephemeral Context Distillation Pipeline (JIT Generation)

Mechanism:

This path confines mathematical text compression entirely to the runtime generation of session artifacts, leaving all committed repository files in their standard, human-readable Linguistic Precision markdown.

The primary architectural modification occurs within the compilation pipeline managed by ops/bin/llms and ops/lib/scripts/synthesize.sh.[1] A new mathematical distillation binary (e.g., a Python or Go utility utilizing standard NLP libraries for token-level entropy scoring and LSA reduction [1]) is introduced into the ops/bin/ registry. When ops/bin/llms is invoked to generate llms-core.txt and llms-full.txt from the CONTEXT.md manifest, the raw text stream is piped through this distillation filter. The filter calculates the Shannon entropy of the token streams, drops predictable linguistic noise, and outputs a highly compressed, high-density prompt

payload.

Simultaneously, ops/bin/open is patched so that the auto-generated session prompt (storage/handoff/OPEN-*.txt) undergoes prompt compression [1] before being handed to the Integrator or Contractor AI.[1] Because ops/bin/open writes artifacts to storage/handoff/ and does not mutate tracked canon files [1], this compression does not trigger Git state dirty flags.

Pros:

- **Zero Canon Drift:** The constitutional documents (PoT.md), the history ledger (SoP.md), and the active contract (TASK.md) remain mathematically un-mutated on disk, ensuring the Operator maintains total visibility and the Relatability Mandate is preserved.[1]
- **Template Hash Integrity Guaranteed:** Because the compression occurs solely in the ephemeral compilation stage and only affects generated .txt artifacts [1], tools/lint/dp.sh and tools/lint/schema.sh will not trigger structural hash failures.
- **Maximum AI Efficiency:** Contractor agents receive vastly smaller prompt payloads, eliminating attention drift and accelerating execution cycles without violating the immutable DP contract.[1]

Cons:

- **Pipeline Latency Introduction:** Adding an SVD/LSA or entropy-scoring calculation layer to ops/bin/compile or ops/bin/llms will inherently increase the execution time of the Verify and Refresh closeout stages.[1]
- **Asymmetric State Architecture:** The AI views a highly compressed, syntax-stripped version of the repository rules, while the Human Operator views the verbose markdown. This asymmetry could lead to misaligned expectations if the distillation algorithm aggressively strips a nuanced structural requirement (like a specific bash flag requirement).
- **Dependency Bloat:** Requires adding mathematical processing libraries (e.g., Python numpy, scipy) to the environment where Stela automation runs, which currently relies heavily on native bash utilities (awk, grep, rg).[1]

Risk Level: Moderate. The implementation is safely isolated to generation binaries, preventing permanent repository corruption, but the risk of asymmetric state misinterpretation requires rigorous unit testing of the distillation filter.

2.2 Option B: Canon-Level Semantic Density Enforcement (Hard Gate)

Mechanism: This path forces the Feynman-Shannon Paradigm directly into the source code of the repository by integrating it as a hard CI/CD gate. It represents an aggressive extension of Stela's existing PoT.md Behavioral Logic Standards.[1]

Currently, tools/lint/style.sh enforces linguistic precision by scanning tracked markdown for

contractions and matching text against a static JARGON_BLACKLIST.[1] Under Option B, a completely new linter, tools/lint/density.sh, is engineered and wired into the .github/workflows/gates.yml and tools/verify.sh sequences.[1]

This new linter evaluates the semantic density [9] of every newly authored DP block in TASK.md and every generated RESULTS artifact in storage/handoff/.[1] It mathematically parses the submitted text, calculating the ratio of high-information tokens to total string length. If the text falls below a defined Shannon entropy threshold—meaning it contains excessive filler, redundant explanations, or conversational padding—the gate fails, and the system issues a **STOP WORK / Reject PR** response as defined in PoT.md Section 2.[1] The Integrator or Contractor must then rewrite the DP or RESULTS using strict Feynman-mode analogical reduction [12] until the text passes the mathematical threshold.

Pros:

- **Absolute Repository Purity:** Forces the entire repository state—not just ephemeral prompts—to adhere to the maximum signal-to-noise ratio. The ledger (SoP.md) and all specifications under docs/ops/specs/ become ultra-efficient and mathematically dense over time.
- **Native Enforcement Philosophy:** Utilizes Stela's existing enforcement architecture (linters, CI gates, and binary wrappers) without requiring external, opaque AI compression APIs at runtime.[1]
- **Mathematical Accountability:** Provides a quantitative, reproducible score for DP quality and documentation clarity, directly fulfilling the PoT.md mandate that requires "quantitative reporting for deviations from protocol".[1]

Cons:

- **Severe Operator Friction:** Human Operators naturally communicate with lower semantic entropy than optimized machines. Forcing humans to author sidecars (e.g., CLOSING-DP-OPS-XXXX.md) that must pass rigorous mathematical density checks may cripple operational velocity and induce profound workflow frustration.[1]
- **Brittle Heuristic Thresholds:** Determining the optimal entropy threshold for tools/lint/density.sh is exceptionally difficult. If calibrated too aggressively, the linter will reject necessary technical context (such as exact literal file paths or multiline bash command structures) as "statistical anomalies," blocking legitimate work.
- **Conflict with Relatability:** Hard-gating dense text directly opposes the PoT directive for "Universal accessibility via vernacular optimization and anecdotal grounding".[1]

Risk Level: High. Modifying CI gates to enforce strict semantic mathematics across all human- and machine-authored markdown carries a high probability of halting all active work if the heuristic is miscalibrated, trapping the repository in endless Dirty State loops.[1]

2.3 Option C: Autonomous Feynman-Mode Factory Agents and JIT

Skills

Mechanism:

Instead of altering global compilation pipelines (Option A) or creating unyielding static linters (Option B), this path localizes the synthesis into a new, on-demand capability within Stela's existing AI Staffing Protocol.

A new AI Agent definition is drafted (archives/definitions/agent-candidate-XXXX.md) and promoted to the active registry as opt/_factory/agents/R-AGENT-08.md, specifically adopting the persona and system prompt of a "Feynman-Distiller".[12] Furthermore, a new Just-In-Time (JIT) Skill (opt/_factory/skills/s-learn-08.md) is authored, containing the explicit algorithmic logic for Semantic Distillation and Relational Analogy Mapping.[1]

When the Integrator generates a highly complex TASK.md DP block that approaches context limits, it explicitly invokes the s-learn-08 skill in the JIT Skills pointer array. The primary Contractor agent is instructed to first pass the intake parameters through the Feynman-Distiller logic, internally compressing the problem space into its core mathematical analogs.[1] Once the Contractor has reduced the problem to pure signal, it executes the standard workflow (e.g., code editing, linting) and utilizes the same skill to distill its final output, ensuring that the generated storage/handoff/DP-OPS-XXXX-RESULTS.md artifact [1] is mathematically dense, highly explanatory, and devoid of conversational padding.

Pros:

- **Non-Destructive Integration:** Leverages the existing opt/_factory/ promotion architecture and agent orchestration rules without fundamentally altering core foundational binaries like ops/bin/compile or strict gates like tools/lint/dp.sh.[1]
- **Contextual Flexibility:** The Feynman-mode skill is invoked conditionally (JIT-only) based on the specific DP's requirements.[1] Routine, low-complexity tasks do not incur the computational overhead of mathematical distillation.
- **Analogical Superiority:** AI agents prompted explicitly to utilize analogical reasoning and entropy-reduction perform significantly better on complex combinatorial optimization and codebase reasoning tasks than those relying on standard Zero-Shot prompting.[12]

Cons:

- **Compliance Dependency:** This option relies entirely on the Contractor AI faithfully interpreting and executing the s-learn-08 markdown instructions. Because the compression is not mathematically forced by a compiled binary gate, the AI may hallucinate the compression or fail to achieve true semantic density.[1]
- **Verification Difficulty:** Stela's core systemic axiom dictates that "everything the system proves is actually true".[1] Proving that an AI agent successfully applied Shannon entropy rules internally is highly difficult without external mathematical validation scripts

confirming the output density.
- **Skill Registry Clutter:** Introduces highly abstract cognitive heuristics into a skill registry that is primarily designed for concrete tasks like API integration (S-LEARN-03) or security protocols (S-LEARN-05).[1]

Risk Level: Low. Because this approach utilizes existing JIT capability structures and operates within the designated AI payload layer, it poses nearly zero risk to repository stability. However, it risks variable effectiveness depending on the foundational LLM's adherence to the skill prompt.

---

3. Recommendation and Proposal

Based on a comprehensive forensic analysis of Stela's strict architectural invariants, CI enforcement mechanisms, and the theoretical requirements of semantic text compression, **a synergistic hybrid of Option A (Ephemeral Context Distillation Pipeline) and Option C (Autonomous Feynman-Mode Factory Agents) constitutes the optimal, lowest-risk deployment path.**

Option B (Canon-Level Enforcement) must be definitively rejected. Stela's physical laws dictate that the docs/ scope exists to "Explain (manuals and rationale)".[1] Forcing human-facing manuals and the historical SoP.md ledger into mathematically dense, minimal-entropy states fundamentally violates the Relatability Mandate, creating an environment that is hostile to the Human Operator whose approvals are required for all merges.[1]

The hybrid A/C approach achieves the mathematical efficiency of a Feynman-Pure architecture while strictly isolating the compression mechanics to the machine-to-machine interfaces, thereby preserving human readability in the canon surfaces.

3.1 Step-by-Step Deployment Plan

To transition Stela to this new paradigm, the following execution sequence must be orchestrated via the Immutable DP Workflow:

1. **Phase 1: Agent & Skill Promotion (The Cognitive Layer):**
    - Draft and promote a new factory capability: S-LEARN-08: Feynman-Shannon Semantic Distillation using ops/lib/scripts/skill.sh.[1]
    - This skill definition must algorithmically instruct Contractor AIs on how to parse incoming TASK.md objectives using deep system analogical mapping rather than surface-level string matching.[14]
    - It will establish the protocol for outputting RESULTS receipts with zero grammatical noise, adhering strictly to the Generation Mandate [1] while ensuring the Operator can still read the execution logs.

2. **Phase 2: Heuristics Engine Upgrade (The Telemetry Layer):**
    - Augment the existing ops/lib/scripts/heuristics.sh script.[1]
    - While the engine currently measures Pattern Density and high churn via git diff --numstat [1], it will be enhanced with a local function that calculates the Shannon entropy of unstaged diff outputs.
    - This upgrade will flag file diffs that introduce excessive linguistic bloat (low information density), fulfilling the PoT.md requirement for quantitative reporting [1] without acting as a hard CI failure gate that blocks closeout.
3. **Phase 3: Synthesis Engine Distillation (The Pipeline Layer):**
    - Execute a refactor of ops/bin/llms and ops/lib/scripts/synthesize.sh.[1]
    - Implement a deterministic algorithmic filter—specifically Semantic Distillation via regex/text filtering of known low-attention stop words—during the generation of the llms-core.txt and llms-full.txt artifacts.
    - This ensures the discovery entry point (llms.txt) provides maximum mathematical "surprise" per token to incoming agents [1], vastly optimizing the platform's baseline operational speed without altering the underlying markdown files stored in docs/ or ops/.

By deliberately combining the systemic, runtime prompt compression of Option A with the targeted, analogical execution capabilities of Option C, Stela will successfully achieve the mathematical density and cognitive reductionism of a Feynman-Pure Architecture. This deployment mathematically guarantees that the system respects the immutability, auditability, and SSOT axioms enshrined within the Policy of Truth.

Works cited

1. Mathematical Text Compression for AI.md
2. How Claude Shannon Invented the Future | Quanta Magazine, accessed February 21, 2026, https://www.quantamagazine.org/how-claude-shannons-information-theory-invented-the-future-20201222/
3. Entropy (information theory) - Wikipedia, accessed February 21, 2026, https://en.wikipedia.org/wiki/Entropy_(information_theory)
4. accessed February 21, 2026, https://www.machinelearningmastery.com/what-is-information-entropy/#:~:text=We%20can%20calculate%20the%20amount,log(%20p(x)%20)
5. Information content - Wikipedia, accessed February 21, 2026, https://en.wikipedia.org/wiki/Information_content
6. A Mathematical Theory of Communication, accessed February 21, 2026, https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf
7. Prediction and Entropy of Printed English - Princeton University, accessed February 21, 2026, https://www.princeton.edu/~wbialek/rome/refs/shannon_51.pdf
8. Semantic Density for NLPs - Use cases and examples - OpenAI Developer

Community, accessed February 21, 2026, https://community.openai.com/t/semantic-density-for-nlps/893468
9. Semantic Density And Its Impact On LLM Ranking | Geeky Tech, accessed February 21, 2026, https://www.geekytech.co.uk/semantic-density-and-its-impact-on-llm-ranking/
10. Professional Word Document Template User Guide: Elegant Formatting and Efficient Memory Methodology - Oreate AI Blog, accessed February 21, 2026, https://www.oreateai.com/blog/professional-word-document-template-user-guide-elegant-formatting-and-efficient-memory-methodology/7f883240425bb25ec5745dfc239c5bf5
11. Your AI Learning Workspace for Researchers - Memo AI, accessed February 21, 2026, https://www.memo.cards/for-researchers
12. Hivemind: Study by Scrolling - App Store - Apple, accessed February 21, 2026, https://apps.apple.com/fr/app/hivemind-study-by-scrolling/id6737344456
13. Life is a Circus and We are the Clowns: Automatically Finding Analogies between Situations and Processes - ACL Anthology, accessed February 21, 2026, https://aclanthology.org/2022.emnlp-main.232.pdf
14. ARN: Analogical Reasoning on Narratives | Transactions of the Association for Computational Linguistics | MIT Press, accessed February 21, 2026, https://direct.mit.edu/tacl/article/doi/10.1162/tacl_a_00688/124260/ARN-Analogical-Reasoning-on-Narratives
15. ANALOGY RECOGNITION AND COMPREHENSION IN EDITORIALS - eScholarship.org, accessed February 21, 2026, https://escholarship.org/content/qt3718d47j/qt3718d47j_noSplash_6ce21835cb21aa125276e42c5566aed5.pdf?t=sgpwtr
16. Leveraging Large Language Models to Develop Heuristics for Emerging Optimization Problems - arXiv, accessed February 21, 2026, https://arxiv.org/html/2503.03350v1
17. Rethinking LLM-Driven Heuristic Design: Generating Efficient and Specialized Solvers via Dynamics-Aware Optimization - arXiv, accessed February 21, 2026, https://arxiv.org/html/2601.20868v1
18. Leveraging Large Language Models to Develop Heuristics for Emerging Optimization Problems - ResearchGate, accessed February 21, 2026, https://www.researchgate.net/publication/389617524_Leveraging_Large_Language_Models_to_Develop_Heuristics_for_Emerging_Optimization_Problems
19. Flesch–Kincaid readability tests - Wikipedia, accessed February 21, 2026, https://en.wikipedia.org/wiki/Flesch%E2%80%93Kincaid_readability_tests
20. Readability Checker - Originality.AI, accessed February 21, 2026, https://originality.ai/readability-checker
21. When writing content, how important is the readability/Flesch Reading Ease score for you?, accessed February 21, 2026, https://www.reddit.com/r/freelanceWriters/comments/km117b/when_writing_content_how_important_is_the/
22. Beyond Flesch-Kincaid: Prompt-based Metrics Improve Difficulty Classification of Educational Texts - arXiv, accessed February 21, 2026,

https://arxiv.org/html/2405.09482v1
23. Analogical Reasoning: An Algorithm Comparison for Natural Language Processing, accessed February 21, 2026, https://www.researchgate.net/publication/357739282_Analogical_Reasoning_An_Algorithm_Comparison_for_Natural_Language_Processing
24. Semantic Density: Uncertainty Quantification for Large Language Models through Confidence Measurement in Semantic Space - arXiv, accessed February 21, 2026, https://arxiv.org/html/2405.13845v3